Applied Cryptography Assignment 2

Name: Shreyansh Seth
Roll No: 2022484
Email: shreyansh22484@iiitd.ac.in
Course: CSE546 – Applied Cryptography
Assignment: 2

Note: I am attempting Questions 2, 3 and 6.
Link to Latex Editor File: https://prism.openai.com/?u=6cea93dd-1084-423a-b844-c25ea90c5d7d&pg=1&m=main.tex&d=7
Notation and small conventions used for all questions:

- For RC4, all index arithmetic is modulo 256. In code, I used `& 255` as an equivalent of `mod 256`.
- For RC4 key-length constraint, I enforced **4–16 bytes** (so the given test key `"Test"` of length 4 bytes is valid). For the bias experiment, I used strictly 16-byte keys as required.
- For Trivium, I used an 80-bit key and 80-bit IV represented as 20-hex-character strings, and I performed the standard 1152-cycle initialization phase before generating keystream bits.
- Note: In my Trivium code, the Python array is 0-indexed, so `State[65]` corresponds to s_{66}, `State[161]` corresponds to s_{162}, etc.
- All source code used for this assignment is available in the attached Jupyter notebook `codes.ipynb`. Each part of questions 2 and 3 are in the jupyter notebook and can be ran by running the whole notebook(no need to download any libs as only standard libs are used).

Question 2 (RC4 Stream Cipher: Implementation and Statistical Bias Analysis)

Problem Given:

- Implement RC4 using KSA and PRGA, then perform encryption/decryption by XOR with keystream.
- Test case: Key (hex) `0x54657374` (ASCII `"Test"`), Plaintext `"Hello World"`. Print first 16 keystream bytes (hex) and verify decryption recovers plaintext.
- Verify RC4 second-byte bias: generate 50,000 random 16-byte keys, record the second keystream byte, count how often it equals 0, and report observed probability. Compare with 1/256 and 2/256.

My Approach:

RC4 is a stream cipher which generates a pseudorandom keystream and XORs it with the plaintext. Even though RC4 is simple and fast, it has well-known weaknesses. In this question, I implemented RC4 using the Key Scheduling Algorithm (KSA) and the Pseudo-Random Generation Algorithm (PRGA), and then I experimentally verified the known **second-byte bias** where the second output byte becomes 0 with probability close to $\frac{2}{256}$ instead of the uniform $\frac{1}{256}$.

Part A: Basic RC4 Implementation

A.1 RC4 components (pseudo-code)

RC4 maintains a permutation array $S[0..255]$ and two indices i and j.

(i) Key Scheduling Algorithm (KSA). Goal: Scramble the initial identity permutation using the key.

```
KSA(Key):
  S = [0,1,2,...,255]
  j = 0
  for i in 0..255:
    j = (j + S[i] + Key[i mod keylen]) mod 256
    swap(S[i], S[j])
  return S, i=0, j=0
```

(ii) Pseudo-Random Generation Algorithm (PRGA). Goal: Generate keystream bytes.

```
PRGA(S, i, j, n):
  for t in 1..n:
    i = (i + 1) mod 256
    j = (j + S[i]) mod 256
    swap(S[i], S[j])
    K[t] = S[(S[i] + S[j]) mod 256]
  return K
```

(iii) Encryption / Decryption. RC4 encryption is XOR with the keystream:

$$C[t] = P[t] \oplus K[t].$$

Decryption is the same XOR operation again:

$$P[t] = C[t] \oplus K[t].$$

A.2 Test case and output (from my run)

Key (hex): `0x54657374` (ASCII: `"Test"`)
Plaintext: `"Hello World"`
I printed the first 16 keystream bytes in hexadecimal, ciphertext in hexadecimal, and verified that decryption recovers the original plaintext.

```
Original Key (hexadecimal): 54657374
Original Key (ASCII): Test
Original Plaintext: Hello World

First 16 Keystream Bytes (hexadecimal): 7a6e4d48423b0e4a74b7bb08539b9bb5
CipherText (hexadecimal): 320b21242d1b592506dbdf
Decrypted PlainText: Hello World

Whether Original PlainText Matches Decrypted PlainText: True
```

Figure 1: RC4 test run output.

Part B: Second-Byte Bias Experiment

B.1 Goal and expected values

The textbook discusses that RC4 has a bias in its early keystream bytes. In particular, the second keystream byte equals 0 about twice as often as expected from a uniform random byte.

Reference probabilities:

$$\Pr[\text{uniform byte} = 0] = \frac{1}{256} \approx 0.00390625$$

$$\Pr[\text{RC4 2nd byte} = 0] \approx \frac{2}{256} \approx 0.0078125$$

B.2 Experiment procedure (pseudo-code)

In my experiment, for each random key I ran the PRGA for the full message length (I used the same message length as Part A), and then I recorded only the second keystream byte for checking the bias.

```
Experiment(N=50000, m):
  countZero = 0
  repeat N times:
    key = random 16-byte key
    S = KSA(key)
    K = PRGA(S, i=0, j=0, n=len(m))   # full keystream for message length
    if K[2nd byte] == 0: countZero++
  observed = countZero / N
  print countZero and observed
```

B.3 Results (my run)

```
Trials Done: 50000
Message Length Used (bytes): 11
Second Byte Being 0 Counts: 394
Observed Probability: 0.00788

Reference Uniform Random Probability: 0.00390625
Reference RC4 Theoretical (approx): 0.0078125
```

Figure 2: RC4 second-byte bias experiment output.

From the output:

$$\widehat{p} = \frac{394}{50000} = 0.00788$$

This value is much closer to $\frac{2}{256} \approx 0.0078125$ than to $\frac{1}{256} \approx 0.00390625$, which confirms the existence of the second-byte bias experimentally.

B.4 Analysis (100–150 words)

My experiment tested RC4 using 50,000 random 16-byte keys and I recorded the second PRGA output byte. I observed 394 zeros, so the measured probability was $394/50000 = 0.00788$. This is almost double the uniform probability $1/256 \approx 0.00391$ and close to the known RC4 bias $2/256 \approx 0.00781$, which shows that RC4's early keystream is not perfectly random. In WEP, many packets are encrypted using related keys because the IV is concatenated with the secret key, and attackers can capture a large number of ciphertexts containing predictable headers. Since RC4's early bytes have bias, an attacker can statistically guess keystream bytes more often than chance, infer plaintext bytes, and over many packets this can also support practical key-recovery attacks.

Question 3 (Trivium Stream Cipher: Design Rationale, Implementation, and Security Analysis)

Problem Given:

- Implement Trivium keystream generator using an 80-bit key and 80-bit IV, describe the 288-bit initialization, and print first 1000 keystream bits after initialization.
- Explain why Trivium uses 1152 initialization cycles and how it helps security.
- Explain why Berlekamp–Massey / linear reconstruction does not work on Trivium.
- Answer short design rationale questions on why Trivium uses three coupled NLFSRs, low-complexity nonlinearity, attack resistance, and one limitation/trade-off.

My Approach:

Trivium is a lightweight stream cipher proposed in the eSTREAM project. Unlike classical LFSR-based stream ciphers, Trivium uses three coupled nonlinear feedback shift registers (NLFSRs) and achieves security through nonlinearity, interaction between the registers, and a long initialization phase. Trivium maintains an internal state of 288 bits split into three registers of lengths $R_1 = 93$, $R_2 = 84$, and $R_3 = 111$. After initialization, each clock cycle updates the state using nonlinear Boolean functions and produces one keystream bit.

3.1 Trivium State Update and Keystream Generation

3.1.1 State representation

I represent the complete 288-bit internal state as a bit array:

$$S = (s_1, s_2, \ldots, s_{288}),$$

where $s_1..s_{93}$ correspond to R_1, $s_{94}..s_{177}$ correspond to R_2, and $s_{178}..s_{288}$ correspond to R_3.

3.1.2 State initialization (80-bit key and 80-bit IV)

Trivium uses an 80-bit key and 80-bit IV. I initialized the 288-bit state in the standard way:

- I loaded the 80-bit key into $s_1..s_{80}$ and set $s_{81}..s_{93}$ to 0.
- I loaded the 80-bit IV into $s_{94}..s_{173}$ and set $s_{174}..s_{177}$ to 0.
- I set $s_{178}..s_{285}$ to 0 and set the last three bits $s_{286} = s_{287} = s_{288} = 1$.

3.1.3 Clock update and output bit

At each clock cycle, I computed:

$$t_1 = s_{66} \oplus (s_{91} \cdot s_{92}) \oplus s_{93} \oplus s_{171},$$

$$t_2 = s_{162} \oplus (s_{175} \cdot s_{176}) \oplus s_{177} \oplus s_{264},$$

$$t_3 = s_{243} \oplus (s_{286} \cdot s_{287}) \oplus s_{288} \oplus s_{69},$$

and the output bit:

$$z_i = s_{66} \oplus s_{162} \oplus s_{243}.$$

After computing t_1, t_2, t_3, I shifted the three registers and inserted t_3 into R_1, t_1 into R_2, and t_2 into R_3 according to the Trivium specification.

3.1.4 Output generation after initialization

Trivium performs 1152 initialization cycles (i.e., 4×288) where output bits are discarded. I ran these 1152 clock cycles first, and then generated and printed the first 1000 keystream bits.

3.1.5 Test run and output (my run)

```
Trivium Input Details:
Key (hex): a1b2c3d4e5f60718293a
IV  (hex): 0f1e2d3c4b5a69788796
Key length (bits): 80
IV length  (bits): 80
Initialization cycles discarded: 1152
Keystream bits requested: 1000

Keystream Summary:
Total bits generated: 1000
Number of 1s: 529
Number of 0s: 471
Fraction of 1s: 0.529
First 64 bits (preview): 1110011100101101101001100010100100111000010010010101100100110001
First 128 bits (hex view): e72da62938495931115be1d67d5c16f9

First 1000 keystream bits after initialization:
1110011100101101101001100010100100111000010010010101100100110001000100010101101111100001110101100111
1101010111000001011011111001000101100000011001100101111111111001101110010101110101111101011110111101
0000001010001110011001101001010110110101011100000101000111010011101100111111101101111110010001010001
1101111101110110011100000011100101001001011110101111001011101011011100110000010101010011011011100111
0001011001101001010110111001001111001111100101001001001010101011011101111110001110101011110100000101100
0111111100011110100011111010010111101100100010010001101001100111110110100010110001011001001111010010
0111001000011110101110001010110110001101010010010101100000101011000101011110100001000111011010101101
1001001110111001000110101011010111101100110011101000100001111100010110011111011100000110110011011101
1000111010110111010110000110010011111001100010001011110010010010101010000000000010110001101111010100
1010001000010011110101101011110101010101000100110001111101011111100011101110100011010110001010111101
```

Figure 3: Trivium test run output (key/IV details and first keystream bits).

3.2 Initialization Phase and Its Security Role

Trivium uses a long initialization phase of 1152 clock cycles because the initial state is highly structured (key bits are placed in one register, IV bits in another, and the rest contains many zeros with only three ones at the end). If I generated keystream bits immediately, the early outputs would depend on the key and IV in a shallow way and would show correlations that can leak information. The 1152-cycle warm-up forces diffusion across all three registers multiple times and increases the algebraic complexity of the internal state. This helps against algebraic attacks because the keystream bits become higher-degree Boolean functions of the key/IV bits, and it helps against correlation attacks because simple linear relationships between keystream and specific state bits get destroyed through nonlinear mixing.

3.3 Resistance to Linear Reconstruction Attacks

If an attacker observes 500 consecutive keystream bits, the Berlekamp–Massey algorithm still cannot be directly used to recover Trivium's internal state because Berlekamp–Massey assumes the sequence satisfies a linear recurrence over $\mathbb{F}_2$ (as in an LFSR). Trivium includes nonlinear feedback terms of the form $(s_a \cdot s_b)$, so the keystream does not follow a bounded-order linear recurrence. These nonlinear terms also cause the algebraic degree of the internal state expressions (in terms of key and IV bits) to grow over time, which makes linear modeling and reconstruction infeasible using a small number of observed bits. In contrast, a single maximal-length LFSR is

purely linear, so observing roughly $2L$ output bits is sufficient to reconstruct the LFSR using Berlekamp–Massey. Similarly, even an XOR combination of multiple LFSRs is still linear overall, so linear reconstruction remains feasible (although the effective linear complexity may be larger).

3.4 Security Design Rationale of Trivium

(1) Why three coupled NLFSRs instead of a single large NLFSR?

Trivium uses three coupled registers to get strong diffusion and interaction while keeping each register simple and hardware-friendly. The coupling ensures that each register influences the others, so the keystream eventually depends on a large portion of the 288-bit state. This design also avoids relying on one complicated NLFSR that could accidentally have a structural weakness. With three coupled parts, the cipher can achieve good security while keeping the update function lightweight.

(2) How does Trivium achieve nonlinearity with very low hardware complexity?

Trivium introduces nonlinearity mainly using a small number of AND operations per clock cycle along with XORs. AND and XOR gates are very cheap in hardware, so the implementation remains lightweight. Even though the nonlinear part per round is small, repeated clocking and coupling spreads this nonlinearity throughout the state. Over many rounds, the keystream becomes a complex nonlinear function of the key and IV bits.

(3) Which classes of cryptanalytic attacks does Trivium aim to resist?

Trivium is designed to resist standard attacks on LFSR-based stream ciphers such as correlation attacks and linear reconstruction attacks. The nonlinear feedback and register coupling also increase resistance to algebraic attacks because it becomes difficult to express keystream bits as low-degree equations in the key bits. The long initialization phase further reduces weaknesses in early keystream output and improves diffusion before any keystream is used.

(4) One limitation or trade-off in Trivium's design

A main trade-off is the initialization cost: Trivium requires 1152 warm-up cycles before any keystream can be used. For short messages, this overhead can reduce efficiency compared to designs that start producing output immediately. Another trade-off is that security depends on correct implementation details such as bit ordering and exact tap positions; mistakes in indexing or initialization can lead to incorrect outputs or reduced security margin.

Question 6 (One-Time Pad, PRGs, and Stream Ciphers)

(a) Why perfect secrecy (OTP) is impractical in real systems

The One-Time Pad (OTP) is theoretically ideal because it provides **perfect secrecy**. Informally, perfect secrecy means that the ciphertext reveals zero information about the plaintext, even to an attacker with unlimited computation power.

A formal way to state perfect secrecy (Shannon security) is:

$$\forall m_0, m_1 \in \{0,1\}^\ell,\ \forall c \in \{0,1\}^\ell :\ \Pr[\mathrm{Enc}(K, m_0) = c] = \Pr[\mathrm{Enc}(K, m_1) = c].$$

Equivalently, it implies:

$$\Pr[M = m \mid C = c] = \Pr[M = m]$$

for all messages m and ciphertexts c.

In OTP specifically, encryption is:

$$C = M \oplus K$$

where M is an ℓ-bit message and K is a uniform random ℓ-bit key used exactly once.

Even though OTP is perfectly secure, it is impractical mainly because of key management:

- **Key length equals message length:** To encrypt an ℓ-bit message, I need an ℓ-bit truly random key. In real communication, messages can be huge or continuous (files, streaming, chats), so distributing a key of the same size becomes unrealistic.
- **Key distribution problem:** OTP requires the sender and receiver to share the long key securely in advance. If I already have a secure way to send massive keys, I could have just used that channel to send the message.
- **Key reuse is catastrophic:** OTP security depends on never reusing a key. If I reuse the same key for two messages M_1, M_2, then

  $$C_1 = M_1 \oplus K, \quad C_2 = M_2 \oplus K,$$

  and XORing ciphertexts cancels the key:

  $$C_1 \oplus C_2 = (M_1 \oplus K) \oplus (M_2 \oplus K) = M_1 \oplus M_2.$$

  This leaks a direct relation between plaintexts and often allows full recovery using redundancy of natural language / formats.
- **Need for true randomness at scale:** OTP requires truly uniform random bits in large volume. High-quality randomness at high bandwidth is itself a practical challenge.

So, OTP is "perfect" in theory, but in real-world systems it fails mainly due to massive key storage, distribution, and strict non-reuse requirements.

(b) Using a PRG to build a stream cipher with a short key + computational security

To avoid OTP's long-key problem, modern stream ciphers replace the long truly random key with a short secret seed and expand it into a long keystream using a **pseudorandom generator (PRG)**.

A PRG is a deterministic efficient function:

$$G : \{0,1\}^n \to \{0,1\}^L \quad \text{where } L \gg n.$$

Input: short seed $s \in \{0,1\}^n$. Output: long bitstring $G(s)$ that "looks random".

Constructing a stream cipher from a PRG: Let the message be $M \in \{0,1\}^L$. I choose a random seed $s \in \{0,1\}^n$ as the secret key, generate:

$$K' = G(s) \in \{0,1\}^L.$$

Then encrypt like OTP:

$$C = M \oplus K' = M \oplus G(s).$$

Decrypt similarly:

$$M = C \oplus G(s).$$

This is basically an OTP-like scheme, but the keystream is pseudorandom, not truly random.

What changes: perfect secrecy → computational security: OTP gives information-theoretic security (attacker can be unbounded). PRG-based stream ciphers give computational security, meaning:

- The ciphertext might not be perfectly independent of the plaintext in an absolute mathematical sense.
- But any efficient attacker (bounded to polynomial-time computation) cannot distinguish the PRG output from truly random bits, so they also cannot learn anything useful about the message.

So I rely on the idea: "Even if some tiny information exists in principle, extracting it is infeasible with realistic computational resources." More formally, the security promise becomes: an adversary running in feasible time (polynomial in n) cannot break it with more than negligible advantage.

This is where computational limitations matter:

- If an adversary had unlimited time, they could brute-force the seed s (try all 2^n seeds).
- But if $n = 128$, brute force is 2^{128}, which is computationally infeasible.

Hence, stream ciphers are practical because I store/distribute only a short key s, I generate a long keystream locally, and security relies on the assumption that breaking the PRG is computationally infeasible.

(c) Formal PRG security definition for $G : \{0,1\}^n \to \{0,1\}^{n+1}$

Here, I must define what it means for G to be a secure pseudorandom generator using the standard "two worlds" indistinguishability game.

Step 1: The two experiments (worlds)

Let A be an adversary (a distinguisher) that outputs 1 if it believes it is in the "PRG world", and 0 otherwise.

Experiment 0 (Real / PRG world):

- Choose a uniform seed $s \leftarrow \{0,1\}^n$.
- Compute $y := G(s) \in \{0,1\}^{n+1}$.
- Give y to adversary: $A(y)$.
- Output the bit A's decision.

Experiment 1 (Random world):

- Choose a uniform string $r \leftarrow \{0,1\}^{n+1}$.
- Give r to adversary: $A(r)$.

- Output the bit A's decision.

So the adversary's goal is to distinguish $G(U_n)$ from U_{n+1}, where U_n is uniform over $\{0,1\}^n$ and U_{n+1} is uniform over $\{0,1\}^{n+1}$.

Step 2: Distinguishing advantage

The distinguishing advantage of A is defined as:

$$\mathrm{Adv}_G^{\mathrm{prg}}(A) = \left| \Pr_{s \leftarrow \{0,1\}^n}[A(G(s)) = 1] - \Pr_{r \leftarrow \{0,1\}^{n+1}}[A(r) = 1] \right|.$$

This measures how much better the adversary is than random guessing at telling the worlds apart.

Step 3: Computational constraints and the definition of security

Now I impose that the adversary is efficient. A standard definition is:

G is a secure PRG if for every probabilistic polynomial-time (PPT) adversary A, the advantage is negligible in n.

Formally:

$$\forall \text{ PPT adversaries } A,\ \exists \text{ negligible function } \mathrm{negl}(n) \text{ such that } \ \mathrm{Adv}_G^{\mathrm{prg}}(A) \leq \mathrm{negl}(n).$$

Where "negligible" means:

$$\mathrm{negl}(n) < \frac{1}{p(n)} \quad \text{for every polynomial } p(n), \text{ for sufficiently large } n.$$

So, putting it together:

- Two worlds: PRG output vs truly random string,
- Advantage: difference in probabilities of outputting 1,
- Constraint: adversary must be PPT, and advantage must be negligible.

That is the formal PRG security definition.